

Empaquetamiento, Artefactos e Inmutabilidad

Construcción Determinista, Repositorios (Nexus, OCI) y Principios DevOps

Prof. Alejandro Muñoz Bravo

Ingeniería de Software V • Universidad ICESI — Período 2026-2

1 Empaquetamiento y Construcción de Software

2 Gestión de Artefactos, Versionamiento e Integridad

3 Repositorios de Artefactos: Nexus y Docker Registry

4 Principio de Artefactos Inmutables en DevOps

01

Empaquetamiento y Construcción

Del código fuente al ejecutable verificable, reproducible y optimizado

En ingeniería de software moderna y DevOps:

- **Definición:** Proceso automatizado que toma código fuente humano, resuelve dependencias y produce un binario ejecutable.
- **Superación del build manual:** Elimina la dependencia de configuraciones locales de IDEs (“en mi máquina compila”).
- **Reproducibilidad:** El mismo commit, en cualquier runner de CI, debe generar exactamente el mismo resultado.
- **Regla Fail-Fast:** Cualquier error de compilación, linter o prueba unitaria aborta el pipeline de inmediato.

Fundamento DevOps

El build es la **primera puerta de calidad** (*Quality Gate*). Si el código no construye de forma limpia y desatendida, no puede ingresar a la cadena de valor.

¿Qué es el Empaquetamiento y qué problema resuelve?

¿Qué es el Empaquetamiento?

Consolidación del código compilado, recursos y dependencias en una **unidad atómica distribuible**:

- Transforma archivos dispersos en un único entregable (.jar, .whl, OCI).
- Define un punto de entrada explícito (Main-Class, ENTRYPOINT).
- Permite versionar y firmar el producto completo.

¿Qué Problemas Resuelve?

- **Dispersión:** Evita copiar ficheros sueltos a servidores.
- **Dependencias faltantes:** Empaqueta o fija librerías necesarias.
- **Ambigüedad:** Estandariza qué y cómo debe ejecutarse.
- **Inconsistencia:** Reemplaza pasos manuales por artefactos certificados.

Flujo secuencial estándar ejecutado por los motores de automatización de build:

1. Checkout

- Clonación limpia del commit exacto.
- Descarga del repo Git sin residuos locales.

2. Resolve

- Resolución de dependencias.
- Descarga desde repositorio proxy local.
- Cotejo con lockfile.

3. Compile

- Compilación a byte-code o binario.
- Linters y análisis estático.
- Transpilación de código.

4. Test

- Pruebas unitarias automatizadas.
- Cálculo de cobertura (*coverage*).
- Fallo en test = Abortar build.

5. Package

- Generación de binario (JAR, OCI).
- Inclusión de metadatos de versión.
- Cálculo de hash SHA-256.

Principio Cardinal: Las fases son estrictamente dependientes: no se empaqueta si las pruebas no pasan al 100 %.

Ecosistema Compilado / Tipado

- **Java / JVM:** *Apache Maven* (declarativo XML, ciclo rígido) y *Gradle* (Groovy/Kotlin DSL, incremental).
Producen JARs/WARs.
- **Go / Golang:** Toolchain nativo (*go build*). Produce binarios estáticos ELF autocontenidos sin runtime externo.
- **Rust:** *Cargo* gestiona dependencias (*crates.io*), compilación y tests con máximas garantías de memoria.

Ecosistema Scripting / Web

- **Node.js / TypeScript:** *npm*, *pnpm* o *yarn* con bundlers (*Vite*, *Webpack*, *esbuild*). Resuelven dependencias y empaquetan bundles.
- **Python:** *Poetry*, *pipenv* y *flit*.
Empaquetan distribuibles en formato *Wheel* (*.whl*) con resolución determinista de dependencias.

Maven formalizó el ciclo de vida estándar secuencial e incremental:

Fases de Desarrollo y Test

- **validate:** Verifica la estructura y metadatos del proyecto.
- **compile:** Compila código fuente a bytecode en `target/classes`.
- **test:** Ejecuta pruebas unitarias (JUnit) sin empaquetar.

Empaquetado y Distribución

- **package:** Empaqueta el bytecode en `.jar` o `.war`.
- **verify:** Ejecuta pruebas de integración y chequeos.
- **install:** Copia el artefacto a la caché local (`~/.m2`).
- **deploy:** Publica el binario en el repositorio remoto (Nexus).

Regla Secuencial: Ejecutar `mvn deploy` ejecuta automáticamente todas las fases previas al 100 %.

El **Project Object Model** define:

- **Coordenadas GAV:**
 - groupId: Dominio invertido de la organización.
 - artifactId: Nombre único del módulo.
 - version: Versión SemVer.
- **Packaging:** Formato del entregable (jar, war, pom).
- **Dependencies:** Librerías externas declaradas para compilación y runtime.

Comando CI recomendado:
`mvn clean verify --batch-mode`

pom.xml (Estructura Estándar)

```
<project xmlns="http://maven.apache.org/POM
/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>co.edu.icesi.ingesoft</groupId>
  <artifactId>servicio-pedidos</artifactId>
  <version>1.4.0</version>
  <packaging>jar</packaging>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</
        artifactId>
      <version>3.2.3</version>
    </dependency>
  </dependencies>
</project>
```

En proyectos de software modernos:

- **Dependencias Directas:** Librerías declaradas explícitamente en el archivo del proyecto.
- **Dependencias Transitivas:** Librerías que nuestras dependencias necesitan para operar.
- **Conflicto de Diamante:** Cuando la librería A requiere C (v1.2) y la librería B requiere C (v2.0).
- **Versiones flotantes:** Usar rangos (`~1.2.0` o `latest`) provoca que dos builds del mismo commit descarguen código distinto en días diferentes.

Riesgo Crítico

El **80 % a 90 %** del código de una app moderna proviene de librerías de terceros. Sin control determinista, el sistema es inestable por diseño.

El Problema del Archivo Manifiesto

- Archivos como `package.json`, `requirements.txt` o `build.gradle` suelen admitir rangos o versiones abiertas.
- Si un autor de una librería publica un parche defectuoso, el build del lunes funciona y el del martes falla misteriosamente sin cambios en Git.

La Solución: Archivo Lock

- **Lockfiles:** `package-lock.json`, `poetry.lock`, `gradle.lockfile`.
- Registran la versión exacta (pinning) y el hash de integridad criptográfico de cada librería descargada.
- **Regla Mandatoria:** El lockfile DEBE estar en Git y usarse en CI con comandos deterministas (`npm ci`).

¿Por qué los contenedores revolucionaron el empaquetamiento?

- **Empaquetado Integral:** No solo empaqueta el binario (JAR, Node app), sino también el runtime (JVM, Node), librerías del SO y variables base.
- **Aislamiento de Entorno:** Elimina diferencias entre máquinas de desarrollo (macOS, Windows) y servidores de ejecución (Linux en la nube).
- **Estandarización OCI:** La *Open Container Initiative* define un formato universal para imágenes y distribución, compatible con Docker, Podman y Kubernetes.

Contenedor vs Binario

Un archivo `.jar` requiere que el host tenga la versión exacta de Java. Una imagen OCI contiene todo lo necesario para ejecutarse de forma autosuficiente.

Dockerfile Monolítico (Anti-patrón)

```
FROM maven:3.9-eclipse-temurin-17
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests
# Compiladores y fuentes en disco!
ENTRYPOINT ["java", "-jar", "target/app.jar"]
```

Resultado: Imagen de **850 MB**, compiladores en producción y enorme superficie de ataque.

Dockerfile Multi-Stage (Buena Práctica)

```
# Etapa 1: Construcción (Builder)
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

# Etapa 2: Runtime limpio de producción
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /build/target/*.jar app.jar
USER 1001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Resultado: Imagen de **120 MB**, solo el JRE mínimo, usuario no-root y cero compiladores.

Lo que **SÍ** debe hacerse

- **Builds herméticos:** Dependencias fijadas con lockfiles exactos.
- **Cacheo de capas:** Copiar dependencias antes del código fuente en el Dockerfile.
- **Multi-stage obligatorio:** Separar compilador de runtime de producción.
- **Pruebas unitarias:** Ejecutar tests en cada build antes de empaquetar el binario.

Lo que **NUNCA** debe hacerse

- **Dependencias flotantes:** Usar latest o versiones comodín en librerías.
- **Compilar en producción:** Clonar Git y ejecutar compiladores en servidores vivos.
- **Secretos en capas:** Incrustar contraseñas o tokens en argumentos de build.
- **Saltar pruebas:** Desactivar tests (`-DskipTests`) para acelerar el pipeline.

- 1 Empaquetamiento y Construcción de Software
- 2 Gestión de Artefactos, Versionamiento e Integridad**
- 3 Repositorios de Artefactos: Nexus y Docker Registry
- 4 Principio de Artefactos Inmutables en DevOps

Sección 02

Gestión de Artefactos e Integridad

Diferenciando código fuente de producto: Metadatos, semántica y seguridad criptográfica

Código Fuente (Source Code)

- Texto plano legible por humanos (Java, Python, TS, Go).
- Estado transitorio y maleable: cambia con cada commit o rama.
- Requiere un entorno de compilación, herramientas de desarrollo y dependencias externas para operar.
- **Almacenamiento:** Repositorios Git (GitHub, GitLab).

Artefacto de Software (Artifact)

- Producto binario empaquetado, verificado y testeado.
- **Inmutable y autocontenido:** Una vez generado para una versión, nunca se altera.
- Listo para ser distribuido, auditado y ejecutado directamente en un entorno.
- **Almacenamiento:** Gestores de artefactos (Nexus, OCI).

¿Por qué es vital un Repositorio de Artefactos?

Comparación de flujo de trabajo: el costo de no contar con un repositorio centralizado:

El Caos Artesanal (Sin Repositorio)

Developer (Build local en laptop)



“¿Me pasas el JAR actualizado?”



Envío por Correo / Drive / Carpeta compartida



¿Cuál era el correcto? ¿v2-final-definitivo?

- Sin trazabilidad, sin versionamiento formal y alto riesgo de desplegar binarios no probados en producción.

Flujo Estandarizado (Con Repositorio)

Developer



REPOSITORIO

(Nexus / OCI)

CI / CD



Testing / Staging / Prod

- Fuente única de la verdad (*Single Source of Truth*), inmutabilidad y despliegues 100 % reproducibles.

Los sistemas empresariales operan sobre distintos niveles de abstracción de artefactos:

1. Nivel Librería / Módulo

- **Java:** JARs, WARs, AARs.
- **Node.js:** Tarballs .tgz de npm.
- **Python:** Wheels .whl, sdist.
- **.NET:** Paquetes NuGet .nupkg.

Propósito: Reutilización de código compartido entre aplicaciones y equipos.

2. Nivel Sistema / Contenedor

- **Imágenes OCI:** Capas comprimidas tar.gz con metadata.
- **Linux OS:** Paquetes .deb o .rpm.
- **Cloud Images:** AMIs de AWS, Golden VMs.

Propósito: Unidad atómica e inmutable de ejecución en producción.

3. Nivel Infraestructura

- **Kubernetes:** Charts de Helm (.tgz).
- **laC Modules:** Módulos versionados de Terraform o OpenTofu.
- **Políticas:** Bundles OPA (Rego).

Propósito: Estandarización de patrones de infraestructura como código.

Estándar universal de nomenclatura para artefactos:
`MAJOR.MINOR.PATCH`

- **MAJOR (Ruptura):** Cambios incompatibles con la API anterior. Los clientes deben adaptar su código para actualizar.
- **MINOR (Funcionalidad):** Nuevas características añadidas de forma compatible hacia atrás.
- **PATCH (Corrección):** Corrección de errores y parches de bugs compatibles hacia atrás.
- **Extensiones:** Prereleases y metadatos:
`2.1.0-rc.2+build.89`.

Semántica y Confianza

SemVer permite a desarrolladores y pipelines automatizar parches de seguridad de forma segura sin temor a romper contratos de API en producción.

100%

de incertidumbre e indeterminismo al desplegar con el tag `:latest`

“:latest no es una versión; es solo un puntero mutable que se mueve al último push. Desplegar :latest en producción rompe la reproducibilidad, imposibilita el rollback determinista y transforma los incidentes en pesadillas forenses.”

¿Cómo garantizar que un binario no fue alterado ni corrompido?

- **Digest Criptográfico:** Transforma los bytes del artefacto en una cadena hexadecimal fija de 256 bits (64 caracteres).
- **Efecto Avalancha:** Si un solo bit del código compilado cambia, el hash resultante cambia drásticamente.
- **Despliegue por Digest en Contenedores:** Kubernetes permite referenciar el digest exacto:

`mi-registro.com/mi-app@sha256:7f83b1657ff1fc53...`

- **Verificación Inmutable:** Se garantiza matemáticamente que lo probado en QA es exactamente lo que corre en producción.

Inmutabilidad Real

Las etiquetas son conveniencias para humanos; los **hashes criptográficos (digests)** son la verdadera identidad matemática e inmutable del artefacto.

En un entorno empresarial moderno:

- **Ataques a la cadena de suministro:** Inyecciones maliciosas en repos públicos (typosquatting, robo de credenciales, incidentes en npm).
- **SBOM (Software Bill of Materials):** Inventario formal de todos los ingredientes: dependencias directas, transitivas, versiones, hashes y licencias.
- **Estándares de la Industria:** *CycloneDX* (enfoque de seguridad) y *SPDX* (estándar ISO/IEC 5962).
- **Firmado Digital (Cosign / Sigstore):** Firmar el artefacto y su SBOM en CI con clave privada o arquitectura *Keyless*.

Transparencia de Software

Un SBOM permite responder en segundos: “¿En cuáles de nuestros 200 microservicios en producción está instalada la librería vulnerable de Log4j?”.

Análisis de Composición (SCA)

- Analiza dependencias declaradas contra bases de datos públicas de vulnerabilidades (NVD / CVE).
- Herramientas: *OWASP Dependency-Check, Snyk, SonarQube, npm audit.*
- Detecta librerías con fallos conocidos antes de empaquetar el binario.

Escaneo de Imágenes OCI

- Inspecciona cada capa del contenedor buscando paquetes del SO base vulnerables (OpenSSL, curl, etc.).
- Herramientas: *Trivy, Gype, Clair, AWS ECR Scanning.*
- **Quality Gate en CI/CD:** Bloquea el build si se detecta una vulnerabilidad CRITICAL con parche disponible.

- 1 Empaquetamiento y Construcción de Software
- 2 Gestión de Artefactos, Versionamiento e Integridad
- 3 Repositorios de Artefactos: Nexus y Docker Registry**
- 4 Principio de Artefactos Inmutables en DevOps

03

Repositorios de Artefactos

Almacenamiento centralizado, gobierno, control de acceso y distribución eficiente

¿Por qué Git NO es un Repositorio de Artefactos?

El Diseño de Git

- Diseñado para comparar y almacenar diferencias línea a línea en **texto plano**.
- Cada versión de un binario (JAR de 50MB o imagen de 300MB) se almacena como un blob completo indivisible.
- **Git Bloat:** Clonar el repositorio descarga todo el historial de binarios pasados, haciendo el clon inviable.

El Diseño de un Artifact Manager

- Diseñado específicamente para almacenar y servir archivos binarios pesados.
- Indexación por metadatos (SemVer, checksums, dependencias, arquitecturas CPU).
- Soporta protocolos nativos (Maven REST, npm registry, Docker v2 Registry API).
- Políticas de retención y purga automática de versiones obsoletas.

Los gestores empresariales implementan una arquitectura de 3 tipos de repositorios fundamentales:

1. Hosted Repository

- **Propósito:** Almacén interno privado de artefactos propios.
- **Separación de ciclos:**
 - *Releases:* Inmutables (1.0.0 nunca se sobrescribe).
 - *Snapshots:* Versiones temporales en desarrollo.
- **Seguridad:** Tokens y roles RBAC.

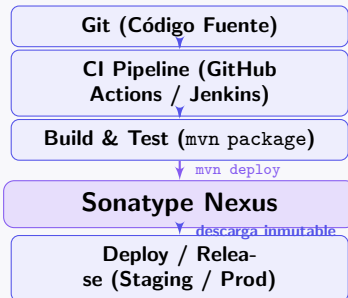
2. Proxy Repository

- **Propósito:** Caché intermediaria de repositorios públicos (Maven Central, npmjs, PyPI, Docker Hub).
- **Beneficios:**
 - **Velocidad:** Descargas a velocidad LAN.
 - **Resiliencia:** Si npm/Hub cae, CI no se detiene.
 - **Ahorro:** Menor consumo de red externa.

3. Group Repository

- **Propósito:** Punto de entrada unificado que combina Hosted y Proxy bajo una sola URL.
- **Simplicidad:** Una única URL en `settings.xml` o `.npmrc`.
- **Prioridad:** Busca primero en Hosted interno, luego en Proxy público.

Ciclo de Vida del Artefacto en Nexus



Aspectos Clave del Flujo

- **Desacople Total:** Los servidores de destino nunca compilan código fuente ni clonan Git.
- **Punto de Depósito Único:** El pipeline de CI es el único actor autorizado para publicar (deploy) en Nexus.
- **Gobernanza y Auditoría:** Nexus registra fecha, autor, pipeline de origen y hash SHA-256.
- **Resiliencia:** El despliegue a producción solo requiere descargar el artefacto pre-empaquetado desde Nexus.

Estándar de la industria en gestión de binarios:

- **Soporte Multiformato Universal:** Gestiona Maven, npm, Docker/OCI, PyPI, NuGet, Helm, RubyGems y archivos Raw.
- **Aislamiento Air-Gapped:** Permite operar pipelines de CI/CD en redes corporativas cerradas sin salida directa a Internet.
- **Cleanup Policies:** Reglas automáticas para borrar snapshots antiguos o imágenes Docker en desuso, evitando saturar el disco.
- **Firewall Integrado:** Bloquea automáticamente paquetes open-source con vulnerabilidades críticas o licencias prohibidas.

Gobierno de Binarios

Nexus actúa como el **fideicomiso de la propiedad intelectual**: todo binario desplegado o consumido está auditado, centralizado y catalogado.

¿Cómo funciona un Docker / OCI Registry?

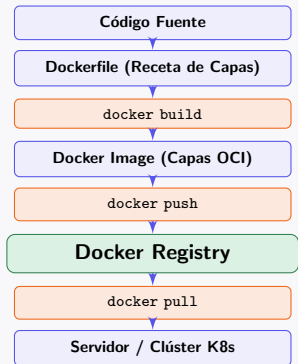
- **OCI Distribution Spec:** Protocolo HTTP API estandarizado para consultar, subir (push) y descargar (pull) imágenes.
- **Content-Addressable Storage:** Cada capa de sistema de archivos es un archivo comprimido (tar.gz) indexado por su SHA-256.
- **Deduplicación Masiva:** Si 50 microservicios usan `alpine:3.19`, los nodos y el registro solo almacenan y transfieren esa capa una vez.
- **Descargas Eficientes:** Si un host ya tiene las capas base, el pull solo descarga la capa de aplicación faltante.

Eficiencia de Red

La arquitectura en capas del OCI Registry permite despliegues en segundos, transfiriendo únicamente el delta modificado en vez de la imagen entera.

Puntos Críticos del Flujo

Pipeline de Contenedorización OCI



- **Construcción en el Runner:** docker build compila y ensambla capas en el entorno efímero del CI.
- **Subida (push):** El runner envía únicamente las capas que no existan previamente en el Registry.
- **Almacenamiento Central:** El Registry indexa manifiestos y almacena los blobs deduplicados.
- **Descarga (pull):** Los nodos del clúster descargan concurrentemente las capas para instanciar los contenedores en producción.

¿Qué es un Manifiesto de Imagen OCI?

El **Manifest** es el documento JSON que actúa como receta de capas de una imagen:

Componentes Esenciales

- **schemaVersion:** Versión formal de la especificación OCI / Docker v2.
- **mediaType:** Tipo MIME del documento y formato de serialización.
- **config:** Descriptor que apunta al blob JSON con ENV, puertos, entrypoint y CPU arch.

Capas y Multi-Arquitectura

- **layers:** Lista ordenada de capas tar.gz con su hash SHA-256 único.
- **Image Index (Multi-Arch):** Manifiesto padre que agrupa arquitecturas bajo la misma etiqueta.

Interpretación del Descriptor:

- **config:** Define cómo se ejecuta el contenedor (CPU arch, ENV, CMD).
- **layers:** Describen el sistema de archivos superpuesto (UnionFS).
- **Deduplicación:** Si dos imágenes comparten una capa, apuntan al mismo digest SHA-256.

manifest.json (Especificación OCI)

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "config": {
    "mediaType": "application/vnd.oci.image.config.v1+json",
    "digest": "sha256:4b9a1f28bc...",
    "size": 7023
  },
  "layers": [
    {
      "mediaType": "application/...layer.v1.tar+gzip",
      "digest": "sha256:d4fc045c0b...",
      "size": 3260901
    },
    {
      "mediaType": "application/...layer.v1.tar+gzip",
      "digest": "sha256:1a82f34e89...",
      "size": 42105
    }
  ]
}
```

Análisis de alcance, formato y convivencia en la arquitectura DevOps:

Criterio	Docker / OCI Registry	Sonatype Nexus Repository
Alcance de Formato	Especializado en imágenes de contenedores OCI/Docker (capas, blobs y manifests).	Multiformato universal: Maven (JAR), npm, PyPI, NuGet, Helm, Docker, Raw.
Ubicación en el Ciclo	Fase de despliegue y runtime (consumido por Kubernetes, ECS, Docker Engine).	Fase de construcción y entrega (resuelve dependencias y almacena releases).
Almacenamiento	Deduplicación estricta a nivel de capas de sistema de archivos por SHA-256.	Almacenamiento organizado por ecosistemas de paquetes y árboles de coordenadas.
Capacidad de Proxy	Pull-through cache de Docker Hub (en implementaciones como Harbor o ECR).	Proxy avanzado de Maven Central, npmjs, PyPI y Docker Hub con reglas de filtrado.
Gobernanza / Firewall	Escaneo de vulnerabilidades en imágenes (Trivy, Clair, Notary/Cosign).	Políticas de licencias open-source, cuarentena de dependencias y auditoría SCA.

Conclusión de Arquitectura: No son mutuamente excluyentes; **coexisten:** Nexus provee las librerías al build, y el Registry sirve las imágenes a producción.

Soluciones Self-Hosted / On-Premise

- **Harbor (CNCF Graduated):** Registro open-source con escaneo Trivy integrado, firma con Cosign, cuotas y replicación entre sedes.
- **Sonatype Nexus / JFrog Artifactory:** Repositorios multiformato con soporte nativo de Docker Registry.
- **Caso de uso:** Soberanía de datos estricta, entornos bancarios o telecomunicaciones.

Servicios Gestionados en la Nube

- **AWS ECR:** Integración nativa con IAM, ECS y EKS; replicación geográfica y escaneo automático.
- **GitHub Container Registry** (`ghcr.io`): Integración natural con GitHub Actions y permisos de repo.
- **Google Artifact Registry / Azure ACR.**
- **Caso de uso:** Equipos cloud-native que eliminan la sobrecarga de operar y respaldar servidores propios.

Mecanismos de control para distribución segura:

- **Mínimo Privilegio:** Runners de CI poseen tokens temporales con permisos exclusivos de `write/push`.
- **ImagePullSecrets:** Clústeres de cómputo solo poseen credenciales de lectura (`read-only`) para descargar imágenes autorizadas.
- **Políticas de Admisión:** Kubernetes rechaza pods de registros públicos no autorizados (Docker Hub personal).
- **Verificación en Runtime:** *Kyverno* o *Cosign* *Gatekeeper* bloquean el despliegue si la imagen no está firmada por el CI.

Zero-Trust Registry

El registro de artefactos es una frontera de seguridad crítica: ningún binario entra sin validación del pipeline, y ningún nodo corre imágenes no firmadas.

- 1 Empaquetamiento y Construcción de Software
- 2 Gestión de Artefactos, Versionamiento e Integridad
- 3 Repositorios de Artefactos: Nexus y Docker Registry
- 4 Principio de Artefactos Inmutables en DevOps**

04

Inmutabilidad en DevOps

Construir una sola vez, promover a través de entornos y desacoplar la configuración

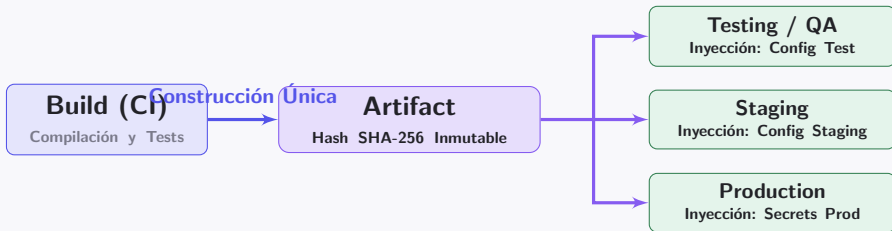
1 Sola Vez

debe compilarse y empaquetarse el código fuente en todo el ciclo de entrega

“Build Once, Deploy to Many Environments. El código fuente se compila y empaqueta una única vez en el pipeline de CI. El artefacto binario resultante es exactamente el mismo que se promueve y ejecuta en Pruebas, Staging y Producción.”

El Flujo de Inmutabilidad: Build Once, Deploy to Many

Un único artefacto binario inmutable alimenta todos los entornos de la cadena de valor:



Garantía Matemática de Calidad

El software aprobado por QA es **bit por bit idéntico** al que entra a Producción. Cero variaciones del compilador ni sorpresas de dependencias.

Velocidad de Despliegue

Promover a producción no requiere compilar: es una simple operación de descarga y arranque en segundos con las variables del entorno.

El anti-patrón más común y peligroso en equipos inmaduros:

- **La práctica errada:** Ejecutar `build --env=qa` para pruebas y otro `build --env=prod` para el release.
- **Dependencias distintas:** El compilador puede resolver versiones parche flotantes diferentes entre builds.
- **Diferencias de flags:** Cambios en optimizaciones de compilación pueden ocultar o generar bugs de concurrencia.
- **Invalidez de pruebas:** Lo que corre en producción jamás fue probado por el equipo de QA.

Invalidez de Pruebas

Si recompilas para producción, estás desplegando software nuevo que jamás pasó por los controles de calidad del entorno de pruebas.

The Twelve-Factor App: Factor III (Configuración)

El Principio de Configuración Externa

- **Regla del Factor III:** Separación estricta de la configuración respecto del código.
- La configuración varía entre entornos (URLs de BD, credenciales, niveles de log).
- El código y las librerías son 100 % idénticos en todos los entornos.
- **Prueba del ácido:** ¿Podrías publicar tu repo en GitHub público sin exponer secretos?

Mecanismos Modernos de Inyección

- **Variables de Entorno:** Estándar agnóstico de SO leído en runtime (`process.env`, `System.getenv()`).
- **ConfigMaps de Kubernetes:** Archivos de configuración desacoplados montados en runtime.
- **Gestores de Secretos:** HashiCorp Vault, AWS Secrets Manager con rotación de llaves.

El Ciclo de Promoción de Artefactos Inmutables

Flujo de promoción continua: el binario es inmutable; solo cambian las configuraciones del entorno:

Fase 1: Build & CI

- Git commit en rama principal.
 - Compilación y pruebas unitarias.
 - Generación de imagen con hash.
 - Push a Nexus / ECR.
- Artefacto: v1.4.0@sha256:...**

Fase 2: Entorno Dev

- Despliegue de imagen inmutable.
- Inyección: Configs de Dev.
- Pruebas de humo automáticas.
- Validación ágil del equipo.

Fase 3: Entorno QA/Stg

- Promoción del **MISMO** hash.
- Inyección: DB Staging / Mocks.
- Pruebas E2E y rendimiento.
- Aprobación formal de release.

Fase 4: Producción

- Despliegue del **MISMO** hash.
- Inyección: Vault / Prod Secrets.
- Blue-Green o Canary.
- Cero sorpresas de compilación.

Garantía Matemática: La certeza de estabilidad en Producción proviene de haber probado el idéntico binario en Staging.

En marzo de 2016, ocurrió uno de los mayores colapsos de la industria:

- **El suceso:** Un desarrollador retiró del repositorio público npmjs una función de 11 líneas llamada `left-pad`.
- **El impacto global:** Miles de proyectos (Babel, React, Facebook, Netflix) que recompilaban en CI descargando de Internet se rompieron en segundos.
- **La causa raíz:** Pipelines sin inmutabilidad que dependían de la disponibilidad en vivo de servicios externos para compilar.

Lección DevOps

1. Nunca dependas de repositorios públicos en vivo durante un despliegue crítico.
2. Utiliza proxies locales (Nexus) con caché inmutable.
3. Despliega artefactos pre-construidos, nunca código dependiente de descargas.

Servidores Tradicionales (Mascotas - Pets)

- Servidores con nombres propios (servidor-pagos-prod).
- Se conectan por SSH para aplicar parches, modificar configs o instalar paquetes en caliente.
- Cada servidor se vuelve único y artesanal con el tiempo.
- **Configuration Drift:** Imposible de reproducir ante una catástrofe; alto riesgo operativo.

Contenedores Inmutables (Ganado - Cattle)

- Contenedores anónimos, idénticos y completamente efímeros.
- **Nunca se modifican en ejecución:** El sistema de archivos raíz es de solo lectura.
- Si hay un bug, se corrige en Git, se construye un nuevo artefacto inmutable y se reemplaza el contenedor.
- Recuperación instantánea ante fallas: destruir y recrear desde la imagen.

Cuadro Comparativo: Paradigma Mutable vs. Inmutable

Dimensión	Paradigma Tradicional (Mutable)	Paradigma DevOps (Immutable)
Construcción	Recompilación en cada entorno (<code>--env=qa</code> , <code>--env=prod</code>).	Construcción única en CI (<i>Build Once, Deploy to Many</i>).
Configuración	Hardcodeada en código o empaquetada en el binario.	Injectada en runtime vía variables de entorno / Vault.
Rollback	Complejo: recompilar o revertir commits en cliente.	Trivial: apuntar el balanceador al digest SHA-256 anterior.
Trazabilidad	Difusa: difícil saber qué versión exacta corre en qué host.	Exacta: cada contenedor tiene un hash SHA-256 inmutable.
Consistencia	Vulnerable al "Configuration Drift" entre entornos.	100 % idéntica entre Dev, QA, Staging y Producción.
MTTR (Reparación)	Horas diagnosticando diferencias en servidores vivos.	Segundos desplegando el contenedor inmutable validado.

05

Síntesis y Discusión

Consolidación conceptual, debate de casos prácticos y conclusiones de la clase

- **1. Build Automatizado:** Proceso determinista con lockfiles y multi-stage builds que separa compilador de runtime.
- **2. Artefactos e Integridad:** El código fuente no se despliega; se gestionan binarios inmutables con SemVer, hashes SHA-256 y SBOMs.
- **3. Repositorios Dedicados:** Sonatype Nexus (Hosted/Proxy/Group) y Docker Registries centralizan y gobiernan la entrega.
- **4. Inmutabilidad y 12-Factor:** “Build Once, Deploy to Many”, configuración externa y contenedores efímeros.

Mensaje Central

La velocidad de entrega en DevOps se logra **industrializando el empaquetamiento y confiando plenamente en la inmutabilidad de los artefactos.**

Analicemos en grupo las siguientes decisiones arquitecturales:

Escenario A: El Frontend Dinámico

Un equipo de React compila una SPA. Como el endpoint backend cambia entre Staging (`api-stg.com`) y Prod (`api.com`), proponen correr `npm run build` dos veces con variables inyectadas en tiempo de compilación.

Pregunta de debate:

¿Qué riesgos introduce esto y cómo diseñarías la solución para cumplir con la inmutabilidad del contenedor Nginx?

Escenario B: El Fallo a Medianoche

Un Dockerfile tiene como directiva inicial `FROM node:alpine` sin especificar versión fija ni digest. Durante 6 meses el pipeline funcionó sin problemas. A las 2 AM en un despliegue de hotfix urgente, el contenedor no inicia.

Pregunta de debate:

¿Qué ocurrió exactamente en el Docker Hub upstream y qué controles fallaron en el pipeline de la organización?

1. **¿Por qué es indispensable `commit`ear el archivo `package-lock.json` o `poetry.lock` en **Git**?**
R/ Para asegurar determinismo: garantizar que cualquier desarrollador o runner de CI descargue exactamente las mismas sub-versiones y hashes de dependencias transitivas.
2. **En Sonatype Nexus, ¿cuál es la diferencia funcional entre un repositorio **Proxy** y uno **Hosted**?**
R/ *Hosted* almacena binarios privados propios de la empresa; *Proxy* cachea y aísla repositorios externos públicos (Maven Central, npmjs, Docker Hub).
3. **Si una aplicación requiere conectarse a diferentes bases de datos en **Dev** y **Prod**, ¿qué debe cambiar?**
R/ Cambian únicamente las variables de entorno inyectadas al contenedor en runtime. La imagen y el binario empaquetado son 100 % idénticos (Twelve-Factor Factor III).

¡Muchas Gracias!

¿Preguntas, dudas o comentarios?

Prof. Alejandro Muñoz Bravo

Ingeniería de Software V • Universidad ICESI
